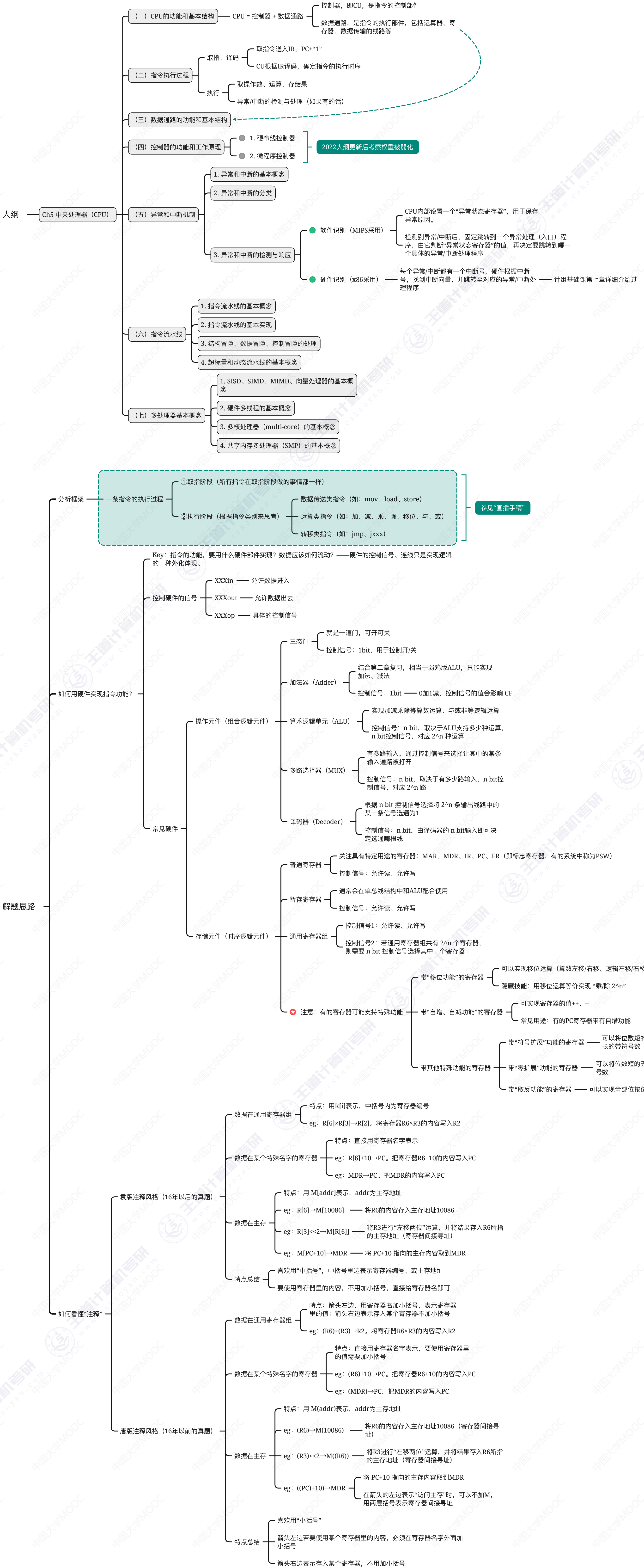


一条指令的硬件实现



一条指令执

① 取指 (所有指令都一样) → 干了什么?

② 执行

数据传送类 (mov, load, store)

运算类指令 (加、减、乘、除、移位、与、或)

转移类指令 (jmp, jxxx)

~~call, ret, trap~~

取指阶段：把PC所指的指令取出来
取到IR

$PC \rightarrow MAR$

$M(MAR) \rightarrow MDR \rightarrow IR$



此“1”为下一条指令(注意指令字长)

$PC + "1"$

实现方式
(硬件)

ALU (加法)

加法器

有“自增功能”的寄存器

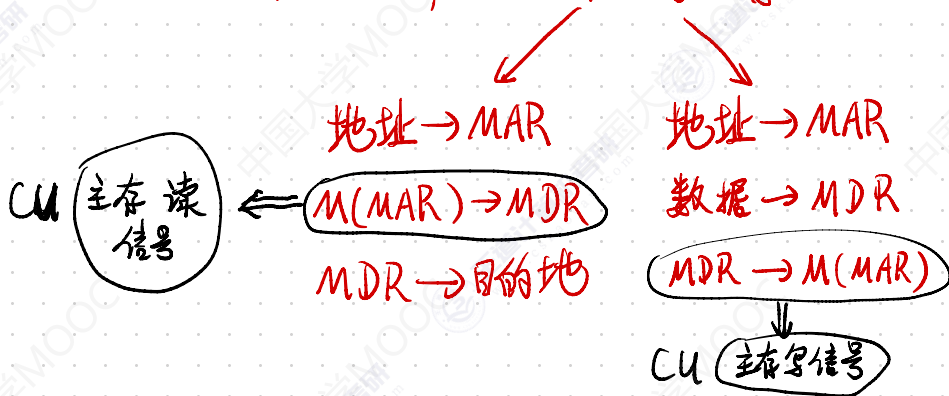
(PC寄存器)

执行阶段：数据传送类

- ① 寄 → 寄
- ② 寄 → 主存
- ③ 主存 → 寄
- ④ 主 → 寄
- ⑤ 主 → 主

⑥ 寄 → 暂存寄存器
↑
某条指令的子步骤

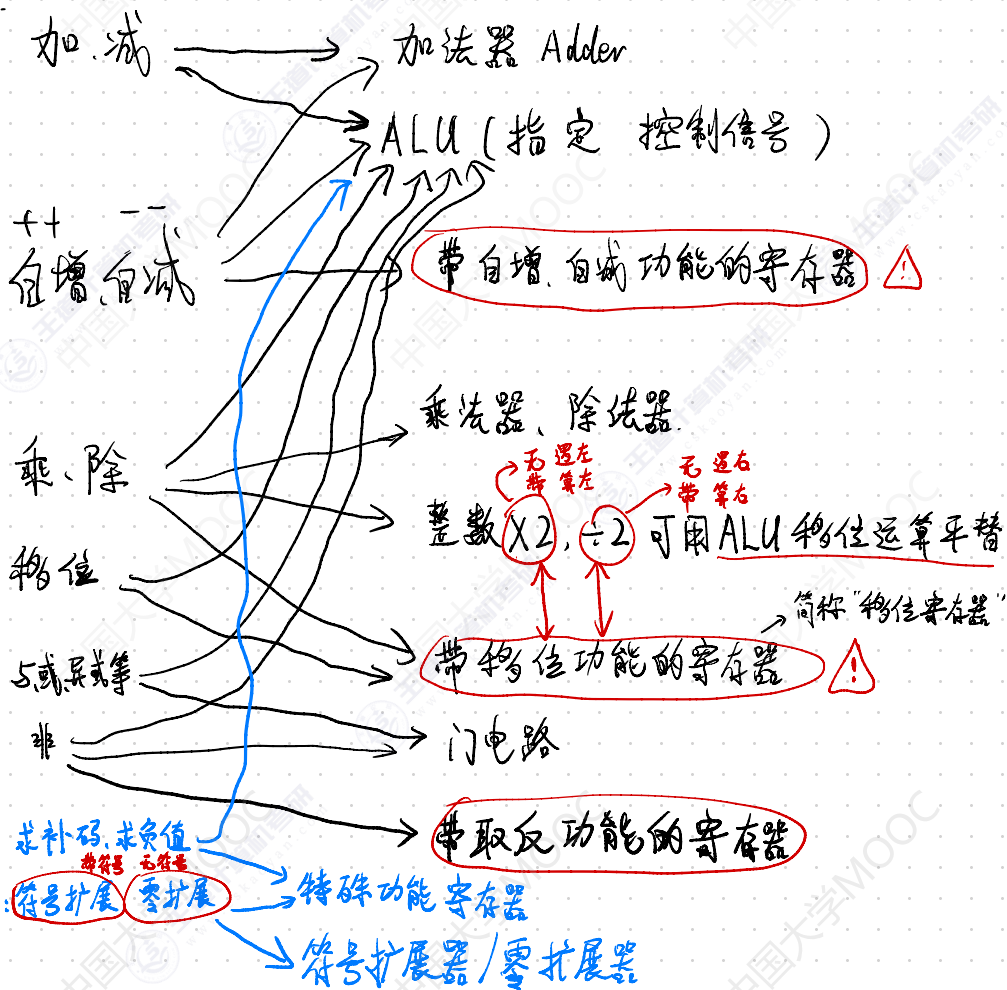
注意1：如果有主存，关注读/写主存



注意2：关注总线占用（总线是临界资源）
（安排控制信号）

执行阶段：运算类指令

可用什么硬件？



www.teskaoyan.com
执行阶段：转移类指令 $\rightarrow$ 改变PC值 (重点题, 13年43题, 4.2.3大8)
(可能)

条件转移 $\rightarrow$ Cmp 前置 (本质是减法 $A-B$, 生成 CF, ZF, OF, SF)

↓
回顾如何生成?

原理：根据 标志位 判断是否转移 (指令中会给 地址码)

如果要转移：①. 注意指令寻址方式

- 相对寻址 $\rightarrow PC + \text{偏移}$
↓
以字为单位
以字节为单位
- 直接寻址 (少见)

RISC (指令长度相同)

② 注意PC的值

- 以字节为单位
- 以指令字为单位

CISC